

Számítógépes grafika 2024 őszi – Deep Sea

Maximum elérhető pontszám: **115 pont**



Ebben a Zárthelyiben a tenger mélyére merészkedünk. Egy kutató tengeralattjáró segítségével kutatunk új élőlények után, amiket robot karral próbálunk megfogni. Tevékenységünket kíváncsian figyelik a gömbhalak.

A víz mélyén mások a fényviszonyok, amit kezelni kell.

Mivel az óceán végtelennek tűnik, nem lepődünk meg a megszokottnál nagyobb számokon. A víz felszíne az $y=0$ sík. Ha az objektum vagy a kamera pozíciójának y értéke negatív, akkor az objektum víz alatt van.

0. Alapok (összesen 2 pont)

Kamera beállítása: a kamera nézze a $(0, -55, 100)$ pontot, és helyezkedjen el a $(0, -50, 105)$ pontban. Világ felfele a $(0, 1, 0)$. **(2 pont)**

Megjegyzés: Emlékezzünk, hogy a kamerát lehet mozgatni föl/le irányban is a Q/E billentyűkkel.

1. Geometriák (összesen 30 pont, minimum 15 szükséges)

1.1. Kétoldalú quad (Σ 6 pont)

A kódban mellékelt **createQuad()** függvénnyel hozzunk létre egy kétoldalú négyzet objektumot. Ezt az objektumot többször fogjuk használni. A négyzet 1 él hosszúságú, és az XY-síkban fekszik.

Talaj

A négyzetet skálázzuk fel úgy, hogy 1000.0 legyen az oldalhossza. Majd forgassuk el, hogy vízszintes legyen. Ezek után toljuk el úgy, hogy a középpontja a $(0, -150.0, 0)$ pontba essen. Használjuk rá az **oceanbottom.png** képet, mint textúrát. **(3 pont)**

Óceán felszín

A négyzetet skálázzuk fel úgy, hogy 1000.0 legyen az oldalhossza, valamint a középpontjuk essen az origóba. Használjuk az **ocean.png** képet, mint textúrát. (3 pont)

1.2. Gömbhalak (Σ 8 pont)

Töltsük be a **PufferFish.obj** fájlt, mint geometriát. Használjuk rá a **PufferFish.png** képet, mint textúrát.

A tengerben több hal is úszik. A mi esetünkben ez legyen $N=5$. Minden halhoz tároljunk egy pozíciót és egy y körüli elforgatás szögét (fokban vagy radiánban, rád van bízva). Kezdetben a forgatás minden halhoz legyen 0. Az i . hal pozícióját (P_i) állítsuk be minden $i \in [0..N-1]$ az alábbi módon:

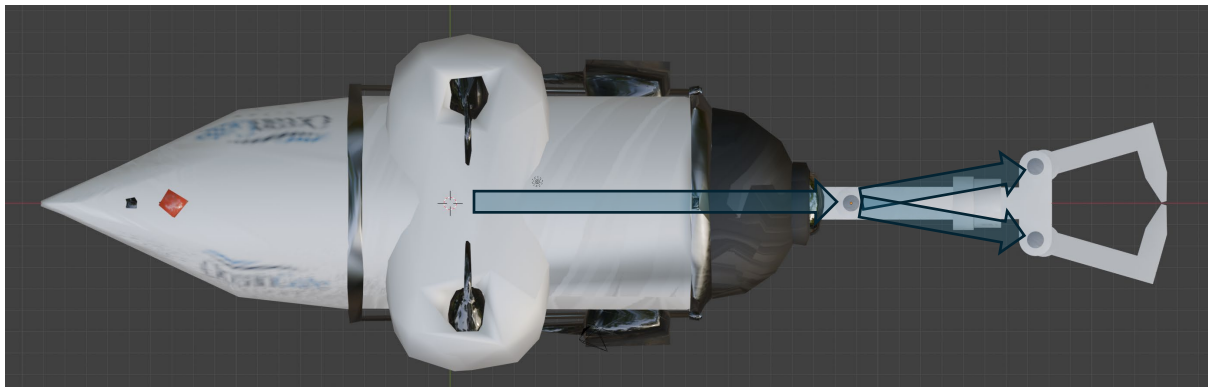
$$P_i = \left(100 \cos\left(2\pi \frac{i}{N}\right), -140 + 130 \frac{i}{N}, 100 \sin\left(2\pi \frac{i}{N}\right)\right)$$

1.3. Tengeralattjáró (Σ 16 pont)

A tengeralattjáró több objektumból lett össze szerelve, de *mindegyik* objektum a **sub.png** textúrát használja. Építsd fel a transzformációkat hierarchikusan a megadottak alapján (lásd ábra is). A megadott transzformációk a „szülő” objektum és a kérdéses al-objektum közötti relatív transzformációt mutatják.

A tengeralattjáró törzse a **sub.obj** fájlban található. A tengeralattjáró pozíciója kezdetben legyen (0,-140,0). (3 pont)

A tengeralattjáróra rá van applikálva egy mozgatható robot kar. A robot kar részeinek pozíciói és orientációi egymáshoz képest hierarchikusan vannak megadva.



A robotkar alapja: **arm.obj**. A robotkar szülő objektuma a *tengeralattjáró*. A saját origóján átmenő Y -tengely körül forgassuk el egy szöggel. Ez a szög legyen állítható -90° és 90° között egy ImGui slider segítségével. Majd toljuk el a (18.75, -3.75, 0) vektorral. (5 pont)

Robot karmok: **claw.obj**. A robot karnak van két fogó karma, de mindkettőt egy objektumból rendereljük különböző transzformációkkal. Mindkettő karomnak a szülője a *ro-*

botkar (arm.obj). Mindkét karom *ugyanazzal* a szöggel van elforgatva saját origóján átmenő Y-tengely körül. Ez a szög legyen állítható 0° és 90° között egy ImGui slider segítségével.

Bal karom: Forgatás után toljuk el a (9, 0, -1.75) vektorral.

Jobb karom: Forgatás után újra forgassuk el a saját X tengelye körül 180°-al. Utána toljuk el a (9, 0, +1.75) vektorral. **(8 pont)**

Megjegyzés: Érdemes használni az ImGui::SliderAngle függvényt szögek beállítására.

2. Shader feladatok (összesen 41 pont, minimum 15 szükséges)

A nap megvilágítja a tengert, és a fény be is hatol a víz alá.

A szintér legyen megvilágítva egy (0,1,0) irányból érkező irány fényforrással. A fragment shader számoljon fény árnyalást a tanult Phong-modell szerint. **(3 pont)**

2.1. Tengerfenék és óceán (Σ 15 pont)

A talajt és az óceánt más módon kell renderelni, mint a többi objektumot.

Mindkét esetben a textúrázáshoz használt textúrakoordináta legyen világkoordináta alapú. Ez azt jelenti, hogy a vertex shaderből beérkező textúra koordináták helyett a beérkező *világ pozíció* x, és z tagját skálázva használjuk, mint textúra koordináta.

$$u \rightarrow \frac{p.x}{150}$$
$$v \rightarrow \frac{p.z}{150}$$

A sampler létrehozásakor az Init()-ben állítsuk az extrapolációs paramétereket GL_CLAMP_TO_EDGE -ről GL_MIRRORED_REPEAT-re. Így ismétlődni fognak a textúrák. **(8 pont)**

Talaj

Talaj rajzolásakor ne használjunk *spekuláris* komponenst (csak akkor ér pontot, ha egyébként a megvilágítás működik). **(2 pont)**

Óceán

Óceán még idő alapján mozog. Adjuk át a fragment shader-nek az eltelt időt. Textúrázásakor adjuk hozzá a világ pozíció x és z tagjához, mielőtt skáláznánk. **(2 pont)**

$$u \rightarrow \frac{p.x + time}{150}$$
$$v \rightarrow \frac{p.z + time}{150}$$

Az óceán rajzolásakor ne használunk fény árnyalást! Csak a „nyers” textúra jelenjen meg. **(3 pont)**

Megjegyzés: Ha mégis használnánk, a lefelé néző négyzet fekete lesz, mivel lefele állnak a normál vektorai.

2.2. Mélység szerinti fényváltozás (Σ 10 pont)

Mikor a fehér fény belép a víztestbe, tovább halad. De a víz sűrűsége nagyobb, mint a levegőé, így a fény hamarabb hal el. Ez az elhalás hullámhossz (tehát szín) függő. Minél mélyebben vagyunk, az elhalás annál nagyobb. Először a vörös szín tűnik el, majd a zöld, és végül a kék.

Egy p pozícióban, a $p.y$ mélységben a fehér fényből „megmaradt” színt mi az alábbi (majdnem fizikailag korrekt) képlettel számolhatjuk:

$$RGB = [e^{0.014 \min(0, p.y)}, e^{0.01 \min(0, p.y)}, e^{0.004 \min(0, p.y)}]$$

Törlési szín

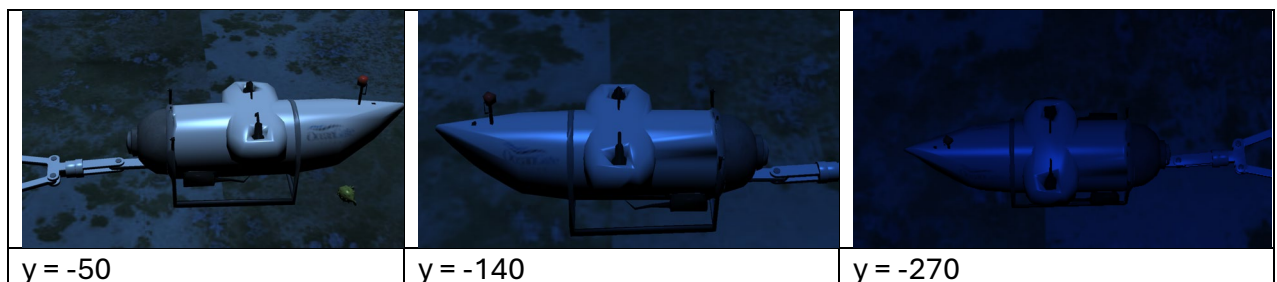
A fenti képletet implementáljuk le. Az Update()-ben hívjuk meg a kamera pozíciójának y koordinátájára, és az eredményül kapott színnel állítsuk be a törlési színt. **(4 pont)**

Megjegyzés: Használható a `glm::exp( glm::vec3 v )` vektorizált függvény is, ami koordinátánként hívja meg az `exp()`-függvényt.

Elhalás víz alatt

A fenti képletet implementáljuk le a fragment shaderben is. Értékeljük ki a fragment világ pozíciójának y koordinátájára. A kapott színnel szorozzuk meg a fragmentben számolt fény árnyalási színét. **(6 pont)**

Megjegyzés: Használható a GLSL `exp( vec3 v )` vektorizált függvénye is, ami koordinátánként hívja meg az `exp()`-függvényt.



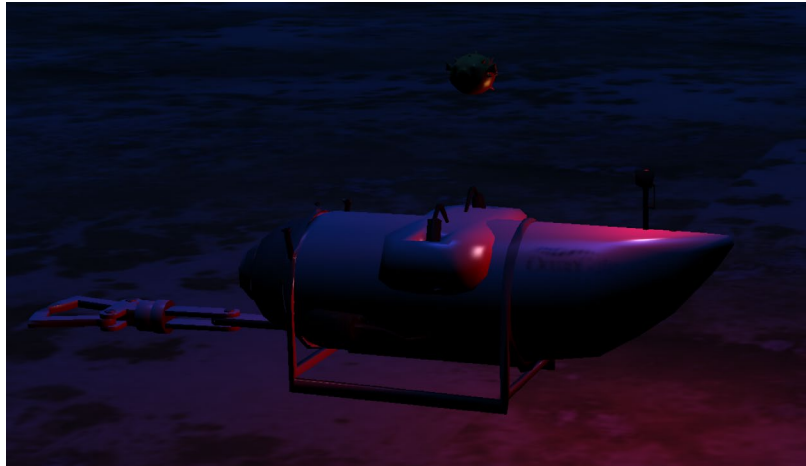
2.3. Jelzőfény (Σ 13 pont)

A tengeralattjárón található egy vörös jelzőfény a többi tengeralattjáró számára. A jelzőfény a hajótest saját koordináta-rendszerében a $(-13, 9, 0)$ pontban található, azaz a tengeralattjáróval együtt mozog.

Hozzunk létre egy piros pontfényforrást. Ez a korábbi irányfényforráson felül szintén hason a színtérben az objektumokra. **(8 pont)**

Ellenben a jelzőfény árnyalására ne legyen hatással a mélységi elhalás. **(2 pont)**

ImGui Checkbox-szal lehessen ki- és bekapcsolni a jelzőfényt **(3 pont)**



3. Egyéb feladatok (összesen 42 pont)

3.1. Halak kiválasztása és módosítása (Σ 10 pont)

A gömbhalat a felhasználó választhassa ki Ctrl+kattintással. A kiválasztásnál tekintsük a halakat 1.75 sugarú gömbnek. (az interakció elő van készítve a kezdő projektben, lásd Update()). Használjuk a meglévő gömbmetszés kódját és kezeljük le azt is, amikor nincs éppen hal kiválasztva.

Ha egy hal éppen ki van választva, akkor ImGui segítségével tudjuk a pozícióját és a forgásszögét módosítani.

3.2. Tengeralattjáró mozgatása (Σ 10 pont)

Merülés

A tengeralattjáró merülési mélységét lehessen változtatni ImGui segítségével 10 és 300 között. **(2 pont)**

Kutatás

A tengeralattjáró a kiválasztott mélységben körözve kutat. A hajó körözzön az origó körül 50 sugarú körben! A hajó mindig nézzen menetirányba. Sebessége kezdetben 10°/s. A szögsebesség legyen állítható ImGui-val. **(5 pont)**

Érjük el, hogy mikor a szögsebességet állítjuk, a tengeralattjáró ne „ugráljon”, hanem folytassa az utat az új sebességgel. **(3 pont)**

3.3. Tengerfenék mélységének állítása (Σ 2 pont)

A tengerfenék mélysége legyen állítható ImGui segítségével 10 és 300 között. (2 pont)

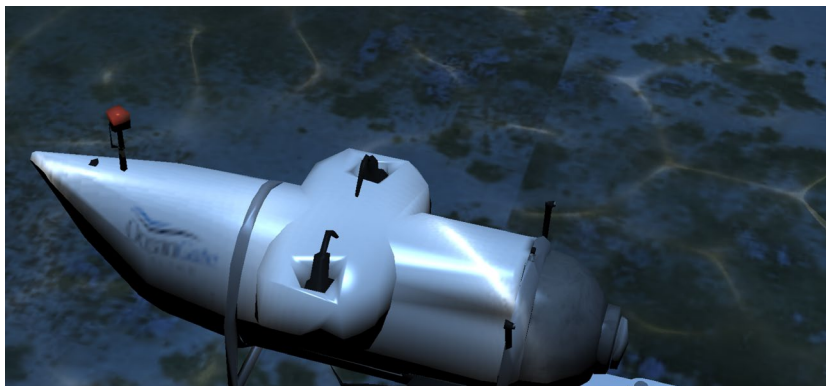
3.4 Óceán és fenék követi a kamerát (Σ 5 pont)

Még a nagy textúrák alól is ki tudunk „szaladni”, és elveszítjük a végtelen óceán illúzióját.

A talaj és óceán négyzeteit rajzoljuk ki úgy, hogy a középpontjainak x és z koordinátái egybe essenek a kamera pozíciójának x és z koordinátaival. Röviden, a négyzetek kövessék a kamerát. (5 pont)

3.5. Fénytörés (Σ 15 pont)

A víz alatt, ha nem vagyunk túl mélyen, látszódnak ahogy a napfény megtörik a hullámokon.



Hozzunk létre egy textúrát a **Caustics.png** képből. Adjuk át *minden* árnyalt objektumnak, mint másodlagos textúrát. A textúra koordináták legyenek ugyanazok, mint az óceánnál (időfüggéssel is).

A fénytörés színét súlyozzuk az alábbiak szerint:

- Ha a normál vektor y koordinátája negatív, nem látszódik a fénytörés, a súlyozás nulla
- Különben a súlyozás legyen $e^{\frac{p.y}{100}}$, ahol p.y a fragment világ pozíciójának y koordinátája.

Kiolvasott és súlyozott értéket adjuk hozzá az árnyalt színhez.